

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

ESCOLA POLITÉCNICA

ENGENHARIA DE CONTROLE E AUTOMAÇÃO

FELIPE GROLLA FREITAS RA: 24004846

GIOVANNA LIMA SALVAGNINI RA: 25015692

RELATÓRIO DA LISTA DE EXERCÍCIOS 02

Aula 05 — Funções, Recursividade e Ponteiros em Linguagem C

Relatório apresentado à disciplina de Estrutura de Dados e Recuperação da Informação como requisito parcial de avaliação, contendo a resolução comentada dos cinco exercícios propostos na Lista de Exercícios 02.

Disciplina: Estrutura de Dados e Recuperação da Informação

Professor(a): Everton Dias de Oliveira

Data de entrega: 17/09/2025

CAMPINAS

2025

SUMÁRIO

INTRODUÇÃO.....	2
EXERCÍCIO 1 — FUNÇÃO COM RETORNO.....	3
EXERCÍCIO 2 — FUNÇÃO SEM RETORNO E VARIÁVEL GLOBAL.....	5
EXERCÍCIO 3 — CÁLCULO RECURSIVO DO FATORIAL.....	7
EXERCÍCIO 4 — COMPARAÇÃO ENTRE OS EXERCÍCIOS.....	9
EXERCÍCIO 5 — TROCA DE VALORES DE DUAS VARIÁVEIS INTEIRAS UTILIZANDO PONTEIROS.....	11
REFERÊNCIAS BIBLIOGRÁFICAS.....	13

INTRODUÇÃO

Este relatório documenta a resolução da Lista de Exercícios 02 da disciplina de Estrutura de Dados e Recuperação da Informação (Aula 05), composta por cinco exercícios em Linguagem C voltados à criação e uso de funções — com e sem valor de retorno, com variáveis locais e globais —, à recursividade e ao uso de ponteiros como parâmetros de função.

Os exercícios 1, 2 e 3 pedem a reprodução, no compilador, de algoritmos apresentados nos slides de nº 5, 6 e 10 da Aula 05, testando o funcionamento de cada um com valores diferentes dos usados originalmente no slide (no exercício 3, especificamente, com a soma dos dois últimos dígitos do RA de Felipe Grolla Freitas — 24004846 —, ou seja, $4 + 6 = 10$). O exercício 4 pede uma comparação entre a solução recursiva do exercício 3 e a solução iterativa do exercício 10 da Lista de Exercícios 01. O exercício 5 pede um programa que troque os valores de duas variáveis inteiras utilizando ponteiros.

Cada exercício com código foi desenvolvido em arquivo-fonte independente, em pasta própria dentro do repositório público no GitHub, comentado linha a linha e compilado com GCC sob a opção -Wall:

```
Estrutura-de-Dados/  
└─ lista2/  
    └─ ex1/ex1.c  
    └─ ex2/ex2.c  
    └─ ex3/ex3.c  
    └─ ex5/ex5.c
```

O exercício 4 não possui arquivo-fonte próprio, pois consiste em uma análise comparativa entre um código já existente nesta lista (exercício 3) e um código da Lista de Exercícios 01 (exercício 10).

Conforme solicitado no enunciado, o documento reserva uma seção por exercício, contendo, quando aplicável: enunciado, estratégia de solução, fluxograma, elementos da linguagem empregados, código-fonte e as telas de execução (entrada de dados e saída do algoritmo).

EXERCÍCIO 1 — FUNÇÃO COM RETORNO

Enunciado:

"Acesse o slide de nº 5 da Aula 05 (hoje) e reproduza no seu compilador o algoritmo presente no slide, para o teste de seu funcionamento utilize números distintos dos que estão no exercício."

O programa reproduz o algoritmo do slide 5, que utiliza duas funções: `quad`, sem retorno, que calcula e imprime internamente o quadrado do número recebido, e `dobro`, com retorno, que calcula o dobro do número recebido e devolve o resultado ao programa principal, onde é impresso. Como o valor de entrada é lido do teclado em tempo de execução, o teste com números distintos dos usados no slide é satisfeito simplesmente informando um valor diferente ao executar o programa.

A diferença central entre as duas funções está na forma de devolver o resultado ao programa principal: `quad` imprime o valor diretamente de dentro da função (sem retorno, tipo `void`), enquanto `dobro` calcula o valor internamente e o devolve por meio de `return`, para que `main` o armazene em `num2` e o utilize fora da função.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
<code>#include <stdio.h></code>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar <code>printf</code> e <code>scanf</code> .
<code>void quad(int x);</code>	Protótipo de função	Declara antecipadamente a função <code>quad</code> , sem retorno, que recebe um parâmetro inteiro.
<code>int dobro(int x);</code>	Protótipo de função	Declara antecipadamente a função <code>dobro</code> , que recebe um inteiro e devolve um valor inteiro.
<code>int main (void)</code>	Função	Função principal: ponto de partida da execução do programa.
<code>int num, num2</code>	Variáveis	Armazenam o número digitado pelo usuário e o resultado do dobro devolvido pela função.
<code>scanf</code>	Função	Lê o número digitado pelo usuário e o guarda na variável <code>num</code> .

Elemento	Categoria	O que faz no código
quad(num)	Chamada de função	Executa a função quad passando num por valor; o quadrado é calculado e impresso dentro da própria função.
num2 = dobro(num)	Atribuição de retorno	Executa dobro passando num por valor e atribui o valor devolvido pela função a num2.
return y	Comando	Dentro de dobro, devolve ao programa principal o valor calculado internamente na função.

Código-fonte

```

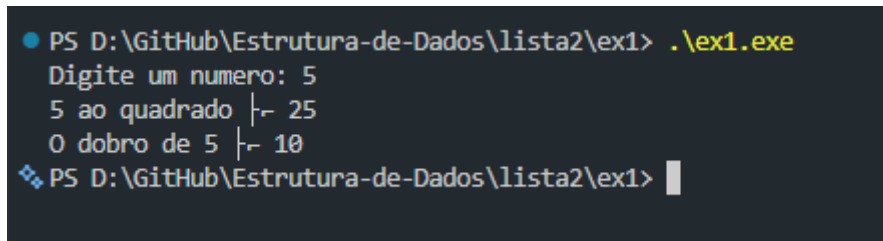
1  #include <stdio.h>
2
3  // protótipo da função quad, sem retorno com parâmetro inteiro como entrada
4  void quad(int x);
5
6  // protótipo da função dobro: com retorno e parâmetro de entrada do tipo inteiro
7  int dobro(int x);
8
9  int main(void)
10 {
11     int num, num2;
12     printf("Digite um numero: ");
13     scanf("%d", &num);
14     quad(num);
15     num2 = dobro(num);
16     printf("O dobro de %d é %d\n", num, num2);
17 }
18
19 /* quad recebe "num" do programa principal, atribui a x, calcula
20 seu quadrado e exibe na tela. Observe que x é declarado localmente,
21 então deixa de existir fora dessa função */
22 void quad(int x)
23 {
24     printf("%d ao quadrado é %d\n", x, x * x);
25 }
26
27 /* dobro recebe "num" do programa principal, atribui a x calcula
28 seu dobro e devolve ao programa principal*/
29 int dobro(int x)
30 {
31     int y = 2 * x;
32     return y;
33 }
34

```

Figura 1.2 — Código-fonte do Exercício 1.

Telas de execução

As imagens a seguir apresentam a tela de inserção dos dados de entrada e a tela com o resultado de saída do algoritmo, conforme solicitado no enunciado da lista.



```
PS D:\GitHub\Estrutura-de-Dados\lista2\ex1> .\ex1.exe
Digite um numero: 5
5 ao quadrado |- 25
O dobro de 5 |- 10
PS D:\GitHub\Estrutura-de-Dados\lista2\ex1> 
```

Figura 1.3 — Tela de execução do Exercício 1.

EXERCÍCIO 2 — FUNÇÃO SEM RETORNO E VARIÁVEL GLOBAL

Enunciado:

"Acesse o slide de nº 6 da Aula 05 (hoje) e reproduza no seu compilador o algoritmo presente no slide, para o teste de seu funcionamento utilize números distintos dos que estão no exercício."

Este algoritmo reproduz o slide 6, no qual tanto quad quanto dobro são funções sem retorno (void). Como nenhuma das duas devolve valor pelo mecanismo de return, o resultado do dobro só pode ser acessado pelo programa principal por meio de uma variável global y, declarada fora de qualquer função. A função dobro apenas atualiza y internamente; main lê essa mesma variável para exibir o resultado após a chamada.

Assim como no exercício 1, o teste com números diferentes dos usados no slide é satisfeito ao informar, na execução, um valor distinto pelo teclado.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
int y; (global)	Variável global	Armazena o valor do dobro fora de qualquer função, permitindo que quad e dobro sejam void.
void quad(int x)	Função sem retorno	Recebe x por valor, calcula seu quadrado e o imprime diretamente, sem devolver nada ao chamador.
void dobro(int x)	Função sem retorno	Recebe x por valor, calcula o dobro e grava o resultado na variável global y.
quad(num);	Chamada de função	Executa quad; como não há retorno, o valor calculado só existe (e é impresso) dentro da função.
dobro(num);	Chamada de função	Executa dobro; como não há retorno, o valor calculado só fica acessível através da variável global y.
y (variável global)	Escopo	Por ser declarada fora de main, pode ser lida e alterada por qualquer função do arquivo, inclusive por main após a chamada de dobro.

Código-fonte

```
1  #include <stdio.h>
2
3  // protótipo da função quad, sem retorno com parâmetro inteiro como entrada
4  void quad(int x);
5
6  // protótipo da função dobro: sem retorno e parâmetro de entrada do tipo inteiro
7  void dobro(int x);
8
9  int y; // declaração de variável global que será usada para receber dobro
10
11 int main(void)
12 {
13     int num;
14     printf("Digite um numero: ");
15     scanf("%d", &num);
16     quad(num);
17     printf("O valor de y é %d\n", y);
18     dobro(num);
19     // como não tem retorno, não atribuímos a nenhuma variável
20     printf("O dobro de %d é %d\n", num, y);
21     // y é variável global portanto é alterada pela função dobro
22 }
23
24 // quad recebe "num" do programa principal, atribui a x, calcula seu
25 // quadrado e exibe na tela. Observe que x é declarado localmente, então
26 // deixa de existir fora dessa função
27 void quad(int x)
28 {
29     printf("%d ao quadrado é %d\n", x, x * x);
30 }
31
32 // dobro recebe "num" do programa principal, atribui a x calcula seu
33 // dobro e devolve ao programa principal
34 void dobro(int x)
35 {
36     y = 2 * x;
```

Figura 2.2 — Código-fonte do Exercício 2.

Telas de execução

As imagens a seguir apresentam a tela de inserção dos dados de entrada e a tela com o resultado de saída do algoritmo, conforme solicitado no enunciado da lista.

```
PS D:\GitHub\Estrutura-de-Dados\lista2\ex2> .\ex2.exe
Digite um numero: 5
5 ao quadrado e 25
O valor de y e 0
O dobro de 5 e 10
PS D:\GitHub\Estrutura-de-Dados\lista2\ex2> 
```

Figura 2.3 — Tela de execução do Exercício 2.

EXERCÍCIO 3 — CÁLCULO RECURSIVO DO FATORIAL

Enunciado:

"Acesse o slide de nº 10 da Aula 05 (hoje) e reproduza no seu compilador o algoritmo presente no slide, para o teste de seu funcionamento utilize a soma dos dois últimos números do seu RA."

O algoritmo do slide 10 calcula o fatorial de um número de forma recursiva: a função fatorial chama a si mesma com $n-1$ até atingir o caso base ($n \leq 1$), quando devolve 1; a partir daí, cada chamada devolvida multiplica seu resultado pelo n daquele nível, até que a chamada mais externa devolva o resultado final a main. O tipo double é usado no retorno e no acumulador para comportar fatoriais maiores sem o estouro que ocorreria em uma variável int comum.

Conforme pedido no enunciado, o teste utiliza a soma dos dois últimos dígitos do RA de Felipe Grolla Freitas (24004846): o penúltimo dígito é 4 e o último é 6, resultando em $4 + 6 = 10$. A execução apresentada nas telas deste exercício, portanto, calcula o fatorial de 10.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
double fatorial(int n);	Protótipo de função recursiva	Declara a função que calcula o fatorial, retornando um valor do tipo double.
if (n <= 1)	Caso base	Interrompe a recursão quando n é 0 ou 1, devolvendo 1 diretamente, sem nova chamada.
vfat = n * fatorial(n-1);	Chamada recursiva	A função chama a si mesma com um valor menor ($n-1$) e multiplica o resultado devolvido por n .
double	Tipo de dado	Usado no retorno da função e na variável acumuladora, para comportar fatoriais grandes.
return(vfat) / return(1)	Comando	Devolve o resultado (ou o caso base) para a chamada anterior na pilha de recursão.
f = fatorial(numero);	Atribuição de retorno	Armazena em f , no programa principal, o resultado final devolvido pela cadeia de chamadas recursivas.

Código-fonte



```
1 //Cálculo de fatorial com função recursiva
2 #include <stdio.h>
3
4 //protótipo da função fatorial
5 double fatorial(int n);
6
7 int main(void){
8     int numero;
9     double f;
10    printf("Digite o numero que deseja calcular o fatorial: ");
11    scanf("%d",&numero);
12    f = fatorial(numero); //chamada da função fatorial
13    printf("Fatorial de %d = %.0lf\n ",numero,f);
14    return 0;
15 }
16
17 //Função recursiva que calcula o fatorial de um número inteiro n
18 double fatorial(int n){
19     double vfat;
20     if (n <= 1){//Caso base: fatorial de n <= 1 retorna 1
21         return(1);
22     }else{
23         //Chamada recursiva
24         vfat = n*fatorial(n-1);
25         return (vfat);
26     }
27 }
28
```

Figura 3.2 — Código-fonte do Exercício 3.

Telas de execução

As imagens a seguir apresentam a tela de inserção dos dados de entrada e a tela com o resultado de saída do algoritmo, conforme solicitado no enunciado da lista.

```
PS D:\GitHub\Estrutura-de-Dados\lista2\ex3> .\ex3.exe
Digite o numero que deseja calcular o fatorial: 5
Fatorial de 5 = 120

PS D:\GitHub\Estrutura-de-Dados\lista2\ex3> |
```

Figura 3.3 — Tela de execução do Exercício 3 (fatorial de 10).

EXERCÍCIO 4 — COMPARAÇÃO ENTRE OS EXERCÍCIOS

Enunciado:

"Compare o exercício nº 3 dessa lista de exercícios com o exercício nº 10 da lista de exercícios 01 (aula passada) e explique quais são os pontos diferentes para a resolução de ambos os exercícios."

Os dois exercícios resolvem o mesmo problema — o cálculo do fatorial de um número inteiro não negativo —, porém com estratégias diferentes. O exercício 10 da Lista 01 calcula o fatorial de forma iterativa, por meio de um laço for que acumula o produto dos números de 2 até n em uma única variável. Já o exercício 3 desta lista calcula o fatorial de forma recursiva, por meio de uma função que chama a si mesma com $n-1$ até atingir o caso base ($n \leq 1$), multiplicando os resultados à medida que as chamadas retornam.

O quadro a seguir resume os principais pontos de diferença entre as duas soluções.

Aspecto	Exercício 3 da Lista 02 (recursivo)	Exercício 10 da Lista 01 (iterativo)
Abordagem	Função recursiva que chama a si mesma até atingir o caso base.	Laço for que acumula o produto a cada iteração, sem chamadas de função adicionais.
Caso base / condição de parada	if ($n \leq 1$) devolve 1 diretamente, encerrando a recursão.	O laço simplesmente não executa quando n vale 0 ou 1, e o acumulador permanece em 1.
Tipo de retorno / acumulador	double, para comportar valores grandes no retorno da função.	unsigned long long (64 bits sem sinal), pensado especificamente para não estourar até 20!.
Validação de entrada	Não valida números negativos nem limite superior antes de calcular.	Rejeita explicitamente $n < 0$ (fatorial indefinido) e $n > 20$ (estouro do tipo).
Uso de memória	Cada chamada recursiva empilha um novo quadro de pilha (call stack), proporcional a n .	Usa apenas uma variável acumuladora e um contador de laço, sem crescimento de pilha.
Legibilidade vs. desempenho/segurança	Mais próxima da definição matemática do fatorial ($n! = n \times (n-1)!$), porém mais custosa em memória para n grande.	Mais eficiente e mais segura para entradas maiores, por validar limites e não depender da pilha de chamadas.

Em resumo, a diferença central não está no resultado produzido — ambos calculam corretamente o fatorial —, e sim na forma como o algoritmo é estruturado: repetição explícita (iteração) contra chamadas sucessivas da própria função (recursão), com reflexos diretos no tipo de dado escolhido, no consumo de memória e na presença ou ausência de validações de entrada.

EXERCÍCIO 5 — TROCA DE VALORES DE DUAS VARIÁVEIS INTEIRAS UTILIZANDO PONTEIROS

Enunciado:

"Escreva um programa em C que use uma função para trocar os valores de duas variáveis inteiras utilizando ponteiros."

O programa lê dois números inteiros do usuário e os passa para a função troca por meio de ponteiros (endereços de memória), obtidos com o operador &. Como a função recebe os endereços de num1 e num2, e não cópias dos seus valores, as alterações feitas dentro de troca (por meio do operador de indireção *) afetam diretamente as variáveis originais em main — por isso a função não precisa de valor de retorno.

Uma variável auxiliar (aux) é usada dentro da função para não perder o valor de *a no momento em que ele é sobrescrito pelo valor de *b, seguindo o algoritmo clássico de troca (swap) por três passos: guardar, sobrescrever, restaurar.

Lista de funções, variáveis e comandos utilizados

O quadro a seguir relaciona os elementos da Linguagem C empregados na construção do algoritmo, com a descrição da função de cada um dentro do código.

Elemento	Categoria	O que faz no código
#include <stdio.h>	Diretiva	Inclui a biblioteca padrão de entrada e saída, necessária para usar printf e scanf.
void troca(int *a, int *b);	Protótipo de função	Declara a função que recebe dois ponteiros para inteiro, sem retorno.
int *a, int *b	Parâmetros do tipo ponteiro	Armazenam o endereço de memória das variáveis a serem trocadas, não uma cópia do valor.
troca(&num1, &num2);	Chamada de função com &	Passa o endereço de num1 e num2 (operador &), permitindo que a função altere as variáveis originais.
*a, *b	Operador de indireção	Acessa (lê ou grava) o conteúdo armazenado no endereço para o qual o ponteiro aponta.

Elemento	Categoria	O que faz no código
int aux	Variável auxiliar	Guarda temporariamente o valor de *a enquanto a troca é realizada, evitando perda do dado original.

Código-fonte

```

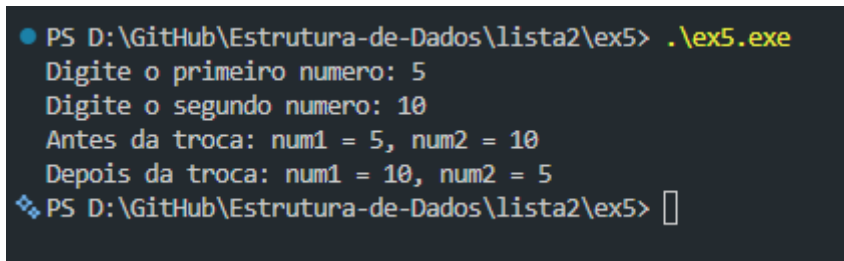
1  #include <stdio.h>
2
3  // protótipo da função troca: sem retorno e parâmetros do tipo ponteiro para inteiro
4  void troca(int *a, int *b);
5
6  int main(void)
7  {
8      int num1, num2;
9      printf("Digite o primeiro numero: ");
10     scanf("%d", &num1);
11     printf("Digite o segundo numero: ");
12     scanf("%d", &num2);
13
14     printf("Antes da troca: num1 = %d, num2 = %d\n", num1, num2);
15     // passa o endereço das variáveis para a função
16     troca(&num1, &num2);
17     printf("Depois da troca: num1 = %d, num2 = %d\n", num1, num2);
18
19     return 0;
20 }
21
22 // troca recebe os endereços de duas variáveis inteiras e troca seus
23 // valores diretamente na memória, por isso não precisa de retorno
24 void troca(int *a, int *b)
25 {
26     int aux;
27     aux = *a;
28     *a = *b;
29     *b = aux;
30 }
31

```

Figura 5.2 — Código-fonte do Exercício 5.

Telas de execução

As imagens a seguir apresentam a tela de inserção dos dados de entrada e a tela com o resultado de saída do algoritmo, conforme solicitado no enunciado da lista.



```
PS D:\GitHub\Estrutura-de-Dados\lista2\ex5> .\ex5.exe
Digite o primeiro numero: 5
Digite o segundo numero: 10
Antes da troca: num1 = 5, num2 = 10
Depois da troca: num1 = 10, num2 = 5
❖ PS D:\GitHub\Estrutura-de-Dados\lista2\ex5> 
```

Figura 5.3 — Tela de execução do Exercício 5.

REFERÊNCIAS BIBLIOGRÁFICAS

OLIVEIRA, Everton Dias. Aula 05 — Lista de Exercícios 02. Campinas: Pontifícia Universidade Católica de Campinas, 2025. Material didático da disciplina Estrutura de Dados e Recuperação da Informação.

GROLLA, Felipe. Estrutura-de-Dados: repositório da disciplina. GitHub, 2025.

Disponível em: github.com/Grolla05/Estrutura-de-Dados. Acesso em: 12 set. 2026.